

Homework 02: But With Youtube Spam

Tulio C. Alberto

2021-03-12T15:00:00 GMT

Contents

0. Data	1
1. Fit a Random Forest model	3
2. Compute and Compare Predictive Performance	4
3. Examine the model	6
Bonus: Use IML to create a PDP+ICE chart	9
4. Summarize	10

0. Data

The data comes from a freely available source here: <http://www.dt.fee.unicamp.br/~tiago//youtubespamcollection/>

Alberto, T. C., Lochter, J. V., & Almeida, T. A. 2015. TubeSpam: Comment Spam Filtering on YouTube. 2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA), 138–143. ieeexplore.ieee.org.

```
load('./youtube_spam_train_test.rda')
yt_ling_train$Segment <- NULL
yt_ling_test$Segment <- NULL
glimpse(yt_ling_train)
```

```
## Rows: 1,200
## Columns: 28
## $ WPS      <dbl> 13.00, 19.50, 8.00, 12.00, 4.50, 9.50, 4.00, 21.00, 10...
## $ WC       <int> 13, 39, 8, 12, 9, 19, 4, 21, 10, 15, 4, 37, 1, 3, 5, 7...
## $ Sixltr   <dbl> 15.38, 10.26, 12.50, 16.67, 11.11, 10.53, 50.00, 0.00,...
## $ Dic      <dbl> 0.00, 20.51, 0.00, 58.33, 22.22, 0.00, 25.00, 0.00, 0....
## $ anger    <dbl> 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ anticipation <dbl> 0.00, 5.13, 0.00, 8.33, 11.11, 0.00, 25.00, 0.00, 0.00...
## $ disgust   <dbl> 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ fear      <dbl> 0.00, 0.00, 0.00, 8.33, 11.11, 0.00, 0.00, 0.00, 0.00,...
## $ joy       <dbl> 0.00, 2.56, 0.00, 8.33, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ negative  <dbl> 0.00, 2.56, 0.00, 16.67, 0.00, 0.00, 0.00, 0.00, 0.00,...
## $ positive  <dbl> 0.00, 2.56, 0.00, 8.33, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ sadness   <dbl> 0.00, 2.56, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ surprise  <dbl> 0.00, 2.56, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ trust     <dbl> 0.00, 2.56, 0.00, 8.33, 0.00, 0.00, 0.00, 0.00, 0.00, ...
```

```
## $ AllPunc      <dbl> 30.77, 23.08, 12.50, 8.33, 22.22, 26.32, 0.00, 14.29, ...
## $ Period      <dbl> 0.00, 0.00, 12.50, 0.00, 11.11, 10.53, 0.00, 14.29, 0....
## $ Comma       <dbl> 7.69, 2.56, 0.00, 0.00, 0.00, 5.26, 0.00, 0.00, 0.00, ...
## $ Colon       <dbl> 7.69, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ SemiC       <dbl> 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ QMark       <dbl> 0.00, 0.00, 0.00, 0.00, 11.11, 0.00, 0.00, 0.00, 0.00,...
## $ Exclam      <dbl> 0.00, 17.95, 0.00, 0.00, 0.00, 10.53, 0.00, 0.00, 20.0...
## $ Dash        <dbl> 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ Quote       <dbl> 0.00, 2.56, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ Apostro     <dbl> 0.00, 2.56, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, ...
## $ Parenth     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ OtherP      <dbl> 15.38, 23.08, 12.50, 0.00, 22.22, 26.32, 0.00, 14.29, ...
## $ url         <dbl> 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, ...
## $ class       <fct> 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, ...
```

```
summary(yt_ling_train)
```

```
##           WPS                WC                Sixltr                Dic
## Min.      : 0.5      Min.      : 1.00      Min.      : 0.00      Min.      : 0.000
## 1st Qu.: 5.0      1st Qu.: 6.00      1st Qu.: 5.26      1st Qu.: 0.000
## Median : 8.0      Median : 9.00      Median : 12.50     Median : 3.925
## Mean     : 12.3     Mean     : 20.23     Mean     : 16.94     Mean     : 17.051
## 3rd Qu.: 13.0     3rd Qu.: 19.25     3rd Qu.: 20.00     3rd Qu.: 22.395
## Max.     :696.0     Max.     :696.00     Max.     :100.00     Max.     :250.000
##      anger      anticipation      disgust      fear
## Min.      : 0.0000      Min.      : 0.000      Min.      : 0.0000      Min.      : 0.0000
## 1st Qu.: 0.0000      1st Qu.: 0.000      1st Qu.: 0.0000      1st Qu.: 0.0000
## Median : 0.0000      Median : 0.000      Median : 0.0000      Median : 0.0000
## Mean     : 0.8927      Mean     : 2.831      Mean     : 0.5759      Mean     : 0.7176
## 3rd Qu.: 0.0000      3rd Qu.: 2.220      3rd Qu.: 0.0000      3rd Qu.: 0.0000
## Max.     :33.3300      Max.     :50.000      Max.     :33.3300      Max.     :33.3300
##      joy      negative      positive      sadness
## Min.      : 0.000      Min.      : 0.000      Min.      : 0.000      Min.      : 0.0000
## 1st Qu.: 0.000      1st Qu.: 0.000      1st Qu.: 0.000      1st Qu.: 0.0000
## Median : 0.000      Median : 0.000      Median : 0.000      Median : 0.0000
## Mean     : 3.142      Mean     : 1.158      Mean     : 4.161      Mean     : 0.8318
## 3rd Qu.: 2.700      3rd Qu.: 0.000      3rd Qu.: 5.298      3rd Qu.: 0.0000
## Max.     :100.000      Max.     :33.330      Max.     :100.000      Max.     :33.3300
##      surprise      trust      AllPunc      Period
## Min.      : 0.0000      Min.      : 0.000      Min.      : 0.00      Min.      : 0.00
## 1st Qu.: 0.0000      1st Qu.: 0.000      1st Qu.: 0.00      1st Qu.: 0.00
## Median : 0.0000      Median : 0.000      Median : 16.13      Median : 0.00
## Mean     : 0.9447      Mean     : 1.796      Mean     : 50.11      Mean     : 10.33
## 3rd Qu.: 0.0000      3rd Qu.: 0.000      3rd Qu.: 33.33      3rd Qu.: 7.14
## Max.     :50.0000      Max.     :50.000      Max.     :5000.00     Max.     :900.00
##      Comma      Colon      SemiC      QMark
## Min.      : 0.000      Min.      : 0.00      Min.      : 0.0000      Min.      : 0.000
## 1st Qu.: 0.000      1st Qu.: 0.00      1st Qu.: 0.0000      1st Qu.: 0.000
## Median : 0.000      Median : 0.00      Median : 0.0000      Median : 0.000
## Mean     : 1.062      Mean     : 5.89      Mean     : 0.9313      Mean     : 1.571
## 3rd Qu.: 0.000      3rd Qu.: 5.13      3rd Qu.: 0.0000      3rd Qu.: 0.000
## Max.     :55.560      Max.     :100.00     Max.     :33.3300      Max.     :100.000
##      Exclam      Dash      Quote      Apostro
## Min.      : 0.00      Min.      : 0.000      Min.      : 0.0000      Min.      : 0.0000
## 1st Qu.: 0.00      1st Qu.: 0.000      1st Qu.: 0.0000      1st Qu.: 0.0000
```

```
## Median : 0.00 Median : 0.000 Median : 0.0000 Median : 0.0000
## Mean : 3.88 Mean : 4.485 Mean : 0.6884 Mean : 0.4642
## 3rd Qu.: 0.00 3rd Qu.: 0.000 3rd Qu.: 0.0000 3rd Qu.: 0.0000
## Max. :99.86 Max. :900.000 Max. :33.3300 Max. :33.3300
## Parenth OtherP url class
## Min. :0 Min. : 0.00 Min. :0.0000 0:559
## 1st Qu.:0 1st Qu.: 0.00 1st Qu.:0.0000 1:641
## Median :0 Median : 14.29 Median :0.0000
## Mean :0 Mean : 42.96 Mean :0.1717
## 3rd Qu.:0 3rd Qu.: 28.57 3rd Qu.:0.0000
## Max. :0 Max. :4400.00 Max. :1.0000
```

```
head(yt_ling_train)
```

```
## WPS WC Sixltr Dic anger anticipation disgust fear joy negative positive
## 1 13.0 13 15.38 0.00 0 0.00 0 0.00 0.00 0.00 0.00
## 2 19.5 39 10.26 20.51 0 5.13 0 0.00 2.56 2.56 2.56
## 3 8.0 8 12.50 0.00 0 0.00 0 0.00 0.00 0.00 0.00
## 4 12.0 12 16.67 58.33 0 8.33 0 8.33 8.33 16.67 8.33
## 5 4.5 9 11.11 22.22 0 11.11 0 11.11 0.00 0.00 0.00
## 6 9.5 19 10.53 0.00 0 0.00 0 0.00 0.00 0.00 0.00
## sadness surprise trust AllPunc Period Comma Colon SemiC QMark Exclam Dash
## 1 0.00 0.00 0.00 30.77 0.00 7.69 7.69 0 0.00 0.00 0
## 2 2.56 2.56 2.56 23.08 0.00 2.56 0.00 0 0.00 17.95 0
## 3 0.00 0.00 0.00 12.50 12.50 0.00 0.00 0 0.00 0.00 0
## 4 0.00 0.00 8.33 8.33 0.00 0.00 0.00 0 0.00 0.00 0
## 5 0.00 0.00 0.00 22.22 11.11 0.00 0.00 0 11.11 0.00 0
## 6 0.00 0.00 0.00 26.32 10.53 5.26 0.00 0 0.00 10.53 0
## Quote Apostro Parenth OtherP url class
## 1 0.00 0.00 0 15.38 0 1
## 2 2.56 2.56 0 23.08 0 1
## 3 0.00 0.00 0 12.50 1 1
## 4 0.00 0.00 0 0.00 0 1
## 5 0.00 0.00 0 22.22 0 1
## 6 0.00 0.00 0 26.32 0 1
```

1. Fit a Random Forest model

Our outcome variable is `class`. Provide a quick look at the distribution of `class` for the training data and test data.

```
count(yt_ling_train, class)
```

```
## class n
## 1 0 559
## 2 1 641
```

```
count(yt_ling_test, class)
```

```
## class n
## 1 0 391
## 2 1 119
```

Comparing the distribution of the outcome variable in training and test, do they look balanced?

The training set looks balanced. The test set looks imbalanced, but it won't impact the model training process.

Fit a random forest model to predict `class`.

```
rf <- randomForest(class ~ ., data = yt_ling_train)
```

Examine the results.

```
print(rf)

##
## Call:
## randomForest(formula = class ~ ., data = yt_ling_train)
##              Type of random forest: classification
##              Number of trees: 500
## No. of variables tried at each split: 5
##
##              OOB estimate of  error rate: 17.33%
## Confusion matrix:
##      0   1 class.error
## 0 467  92  0.1645796
## 1 116 525  0.1809672
```

What is the out-of-box error rate?

OOB estimate of error rate 17.3% showing an effective prediction.

2. Compute and Compare Predictive Performance

Use the `confusionMatrix` function to compute several metrics of predictive performance.

```
rf$confusion[, -3] %>%
  as.table %>%
  confusionMatrix()

## Confusion Matrix and Statistics
##
##      0   1
## 0 467  92
## 1 116 525
##
##              Accuracy : 0.8267
##              95% CI : (0.804, 0.8477)
##              No Information Rate : 0.5142
##              P-Value [Acc > NIR] : <2e-16
##
##              Kappa : 0.6527
##
##  Mcnemar's Test P-Value : 0.1108
##
##              Sensitivity : 0.8010
##              Specificity : 0.8509
##              Pos Pred Value : 0.8354
##              Neg Pred Value : 0.8190
##              Prevalence : 0.4858
```

```
##          Detection Rate : 0.3892
##    Detection Prevalence : 0.4658
##          Balanced Accuracy : 0.8260
##
##          'Positive' Class : 0
##
```

How would you describe the performance of this model?

The accuracy is 82.7%, which is pretty good. The model is more specific than sensitive. The model is somewhat better at predicting non-spam comments than spam comments.

The confusion matrix assumed a 0.50 cutoff for prediction. Now create a ROC plot and compute the AUC for the training set.

```
p <- predict(rf, newdata = yt_ling_train, type = "prob")[,2] %>%
  as.numeric
yt_train_roc <- roc(yt_ling_train$class, p)
```

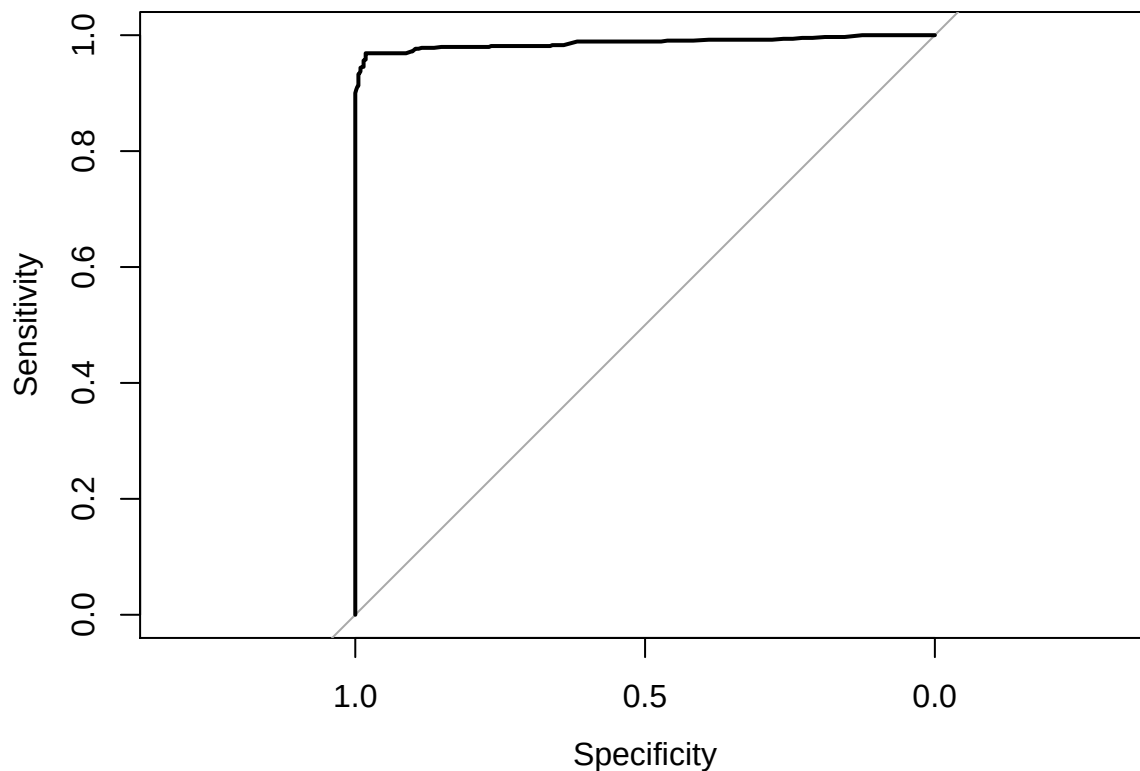
```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc(yt_train_roc)
```

```
## Area under the curve: 0.987
```

```
plot(yt_train_roc)
```

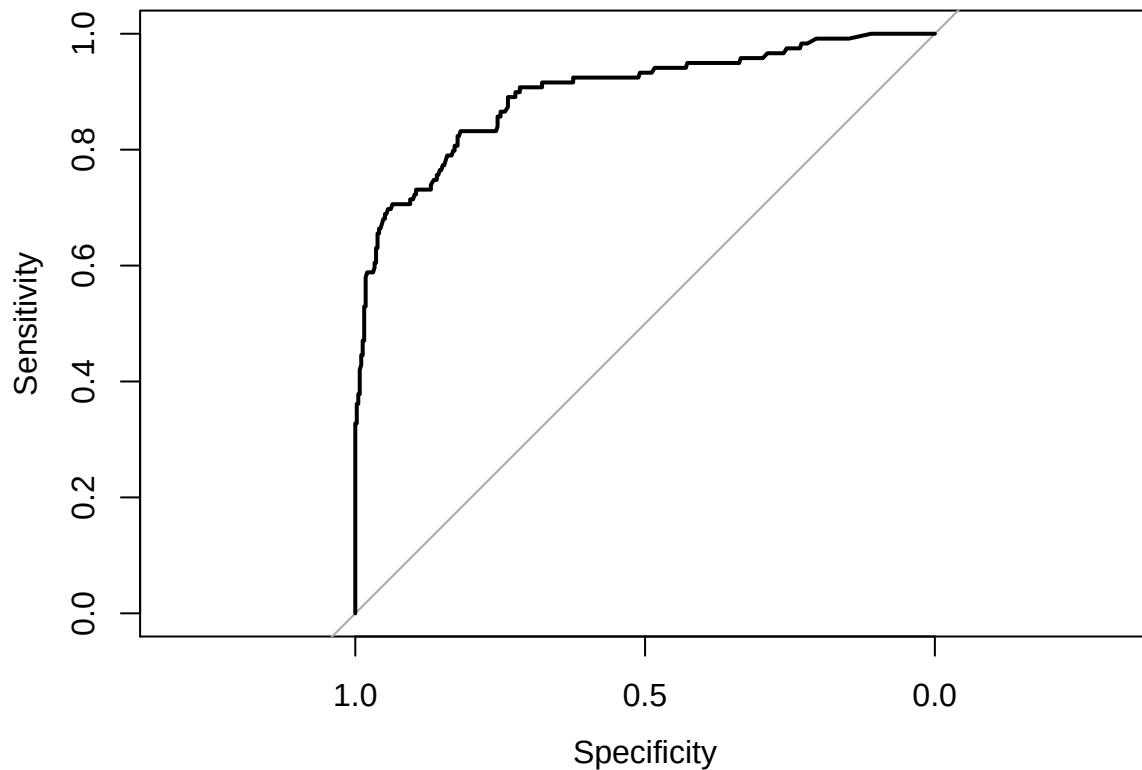


Now create a ROC chart for the test set and compute the test AUC.

```
p <- predict(rf, newdata = yt_ling_test, type = "prob")[,2] %>%
  as.numeric
yt_test_roc <- roc(yt_ling_test$class, p)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
auc(yt_test_roc)
```

```
## Area under the curve: 0.8968
plot(yt_test_roc)
```



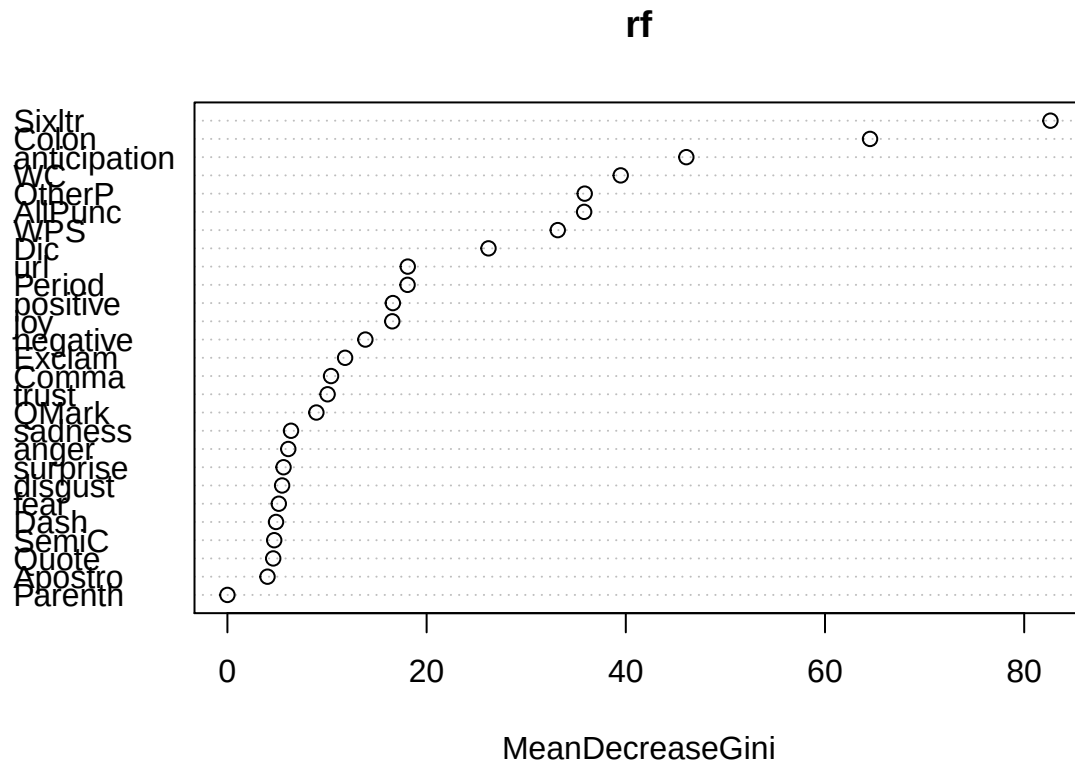
Is the model underfit, overfit, or correctly fit to the data?

The model is somewhat overfit. The test-AUC is 0.09 lower than the train-AUC indicating that the model overfit to the training data.

3. Examine the model

Examine the variable importance of the model.

```
varImpPlot(rf)
```



Make some individual predictions of the model. Choose one case from the data and see what the model predicts for that one person.

```
d <- yt_ling_train[8,]
predict(rf, newdata = d, type = "prob")
```

```
##      0      1
## 8 0.946 0.054
## attr("class")
## [1] "matrix" "array"  "votes"
```

Using the *most important* variable from the plot above, change the value of that variable to something new and make a new prediction for that one case (i.e. set the value to something very small, or very large). How does the prediction change?

```
d$Sixltr <- 10
predict(rf, newdata = d, type = "prob")
```

```
##      0      1
## 8 0.682 0.318
## attr("class")
## [1] "matrix" "array"  "votes"
```

Let's now try and predict the outcome for this case if that important variable was changed from it's minimum value to it's maximum value.

1. Create a grid of at least 100 points from the minimum value to the maximum value.
2. Duplicate the case you used above the same number of times
3. Add the grid of points to the data
4. Predict the outcome using this new fake data and save the predicted probability in the dataset.

```
N <- 1000
g <- seq(min(yt_ling_train$Sixltr),
```

```

      max(yt_ling_train$Sixltr),
      length.out = N)
dn <- d %>%
  mutate(n = N) %>%
  uncount(n) %>%
  mutate(Sixltr = g)
dn$pred <- predict(rf, newdata = dn, type = "prob")[,2]

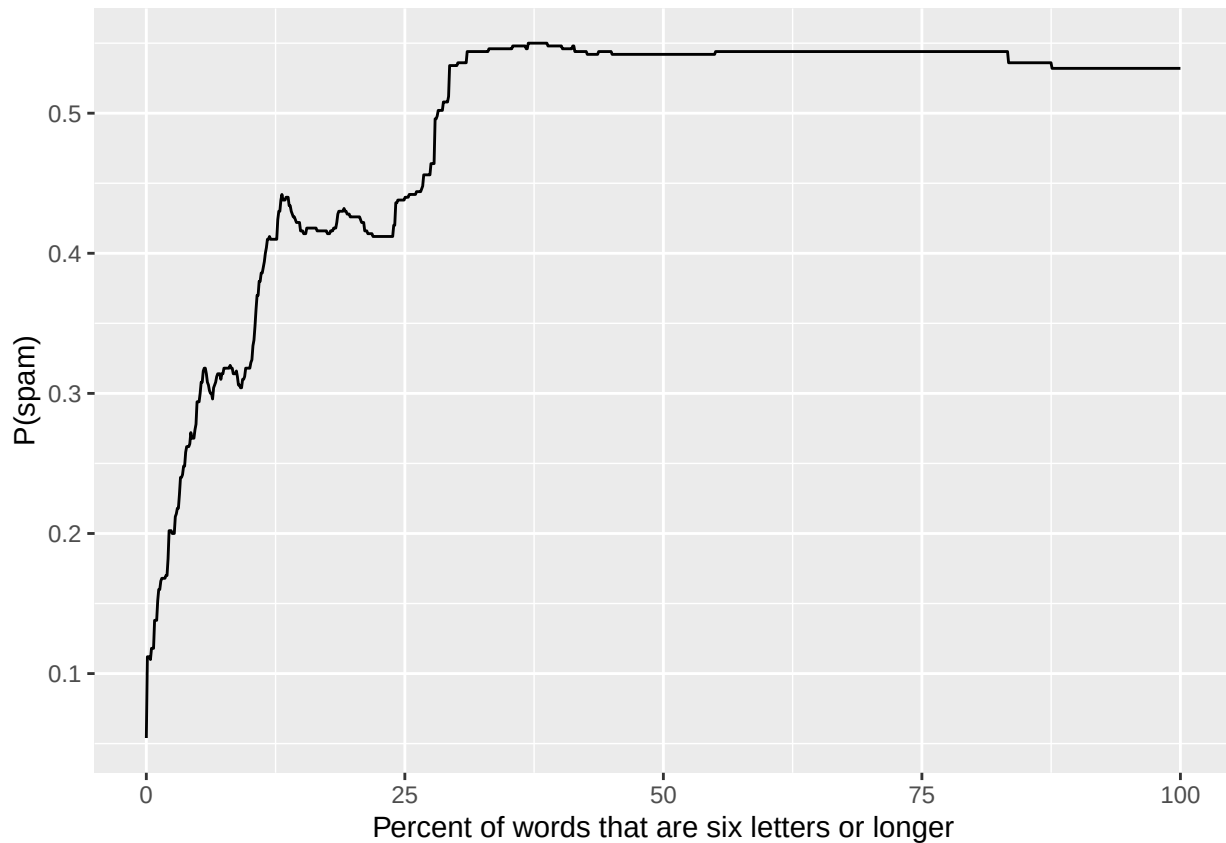
```

Now plot the results of that sequential grid against the predicted probability. How do you see the probability of responding the mailer change in response to the variable? Tip: You can use `coord_cartesian` to change the xlimits to focus on specific areas of detail if you want.

```

ggplot(dn, aes(x = Sixltr, y = pred)) +
  geom_line() +
  coord_cartesian(xlim = c(0, 100)) +
  xlab("Percent of words that are six letters or longer") +
  ylab("P(spam)")

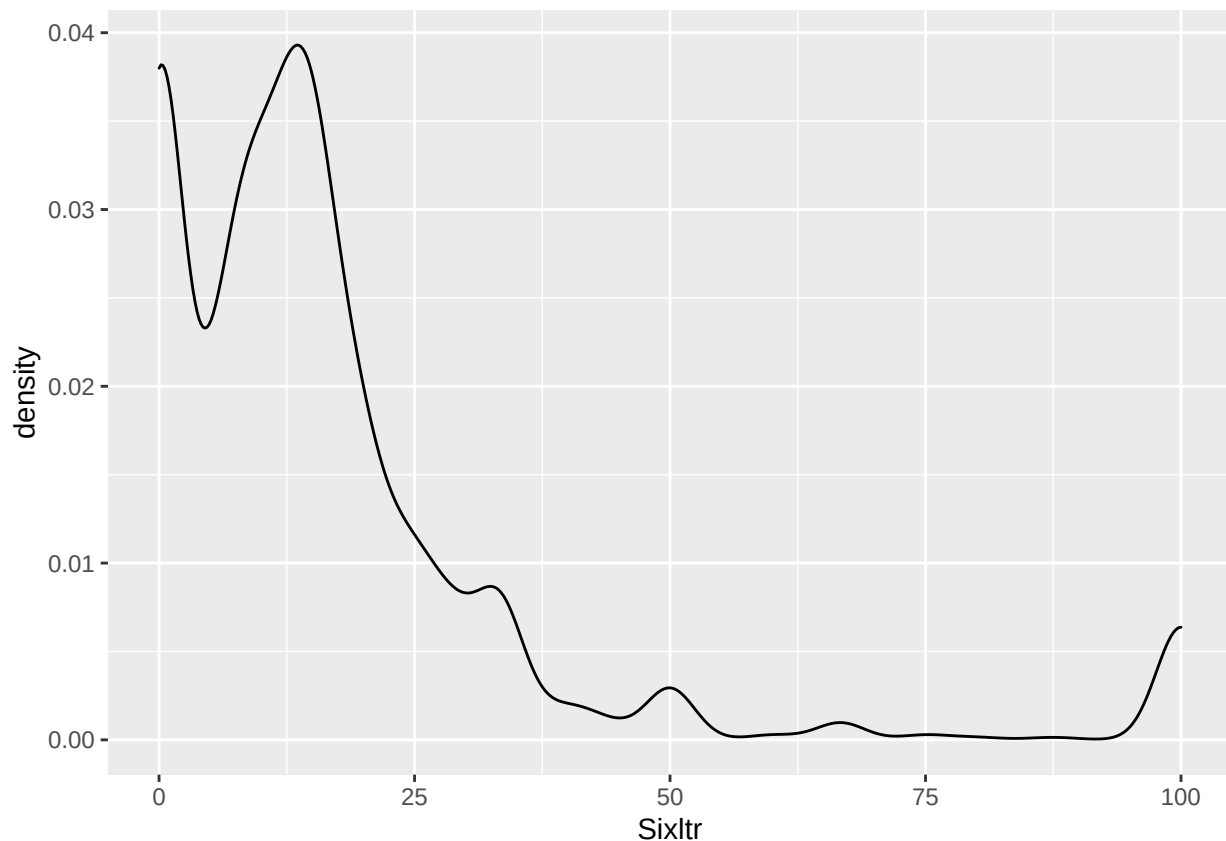
```



```

ggplot(yt_ling_train, aes(x = Sixltr)) +
  geom_density()

```



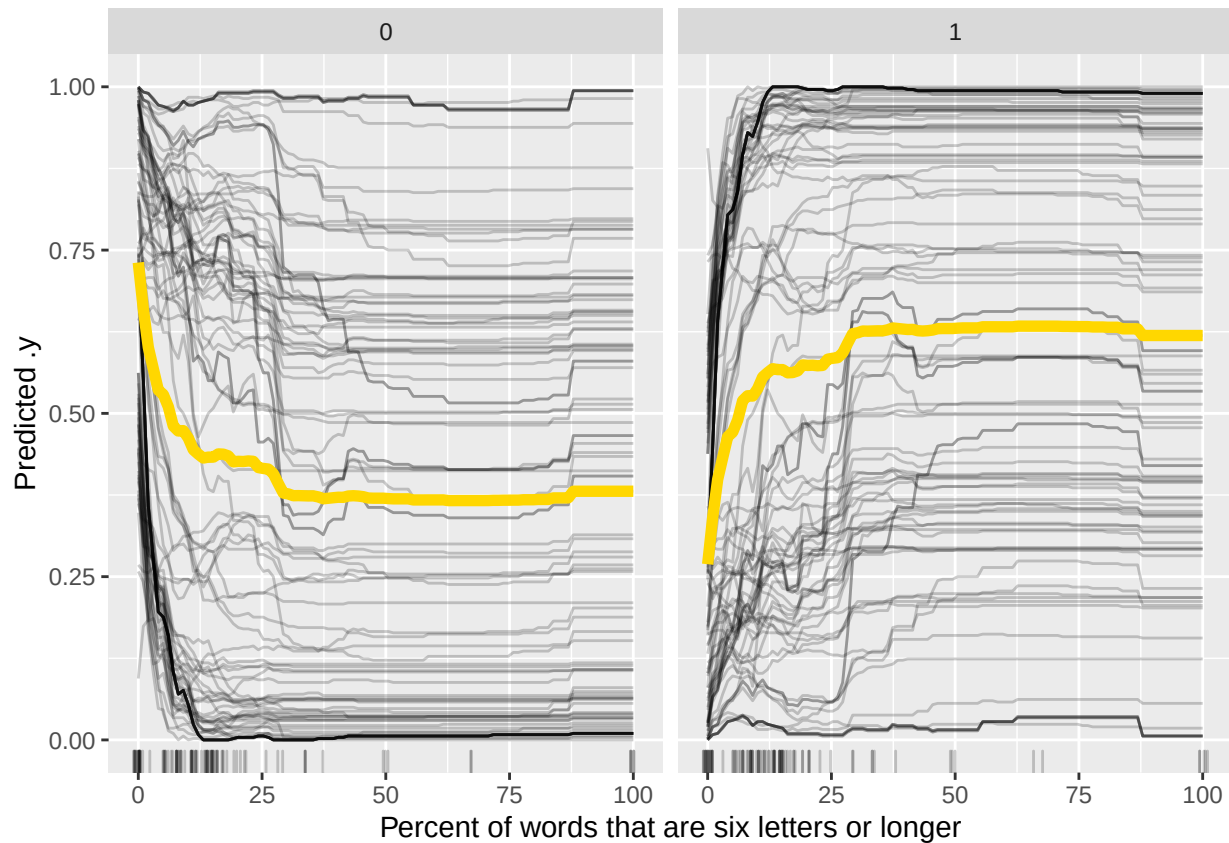
Bonus: Use IML to create a PDP+ICE chart

Use the `iml` package to create an individual conditional expectations combined with partial dependence chart.

It's highly recommended that you only provide the prediction object a subset of the data. I'm not sure the RStudio Cloud instance can take the full data and it will take a very long time. Use `coord_cartesian` to focus the y-limit range to focus the chart to a region where you can observe the effect (it's tiny!).

```
library(iml)
d <- sample_n(yt_ling_train, 100)
yt_pred <- Predictor$new(model = rf, data = d,
  y = d$class, type = "prob")
yt_effect <- FeatureEffect$new(yt_pred,
  grid.size = 100,
  method = "pdp+ice",
  feature = "Sixltr")

yt_effect$plot() +
  # coord_cartesian(ylim = c(0.3, 0.8)) +
  xlab("Percent of words that are six letters or longer") +
  ylab("P(spam)")
```



4. Summarize

What are your thoughts on the impact of these different features on the likelihood for a person to respond to our donation requests? (100-200 words)

To reduce overfitting I would regularize the random forest by trying to find a good mtry parameter value so that the test and train AUC are more similar. This could impact trainin performance, but improve test performance. Real commenters tend to use shorter words. User data about where they post and when. Do they typicaly post comments like this? Are all the same?

Things you could write about:

- What changes would you make to the modeling approach to improve the predictions?
- What recommendations would you make to the organization?
- What other kinds of data would be useful in improving these predictions?